



UNIVERSITI MALAYSIA TERENGGANU
FACULTY OF COMPUTER SCIENCE AND MATHEMATICS

CSE3433

Software Architecture

Cloud-Native Learning Management System (LMS)
(Cloud-Native / Layered Architecture)

Prepared by:

Group No.5

No.	Name	Matric No.
1	NUR MAS QASRINA BINTI MAS JAYA	S74706
2	AIDA HUSNA BINTI MUHAMAD ZAIDI	S76103
3	NURSYAHEERA BINTI MOHD HISHAM	S76276

Prepared for:

DR MOHD NOR HASSAN

COMPUTER SCIENCE (SOFTWARE ENGINEERING) WITH HONOUR
SEMESTER II 2025/2026

Table of Contents

1. INTRODUCTION.....	3
2. PROBLEM DESCRIPTION.....	3
3. SYSTEM REQUIREMENTS.....	4
3.1 FUNCTIONAL REQUIREMENT.....	4
3.2 NON-FUNCTIONAL REQUIREMENT.....	5
4. ARCHITECTURE DESIGN.....	6
5. ARCHITECTURE DIAGRAMS.....	7
5.1 SYSTEM CONTEXT DIAGRAM.....	7
5.2 COMPONENT / SERVICES ARCHITECTURE DIAGRAM.....	8
5.3 INTERACTION DIAGRAM.....	10
5.4 DEPLOYMENT DIAGRAM.....	15
6. TECHNOLOGY STACK.....	17
7. PROTOTYPE IMPLEMENTATION.....	17
8. ARCHITECTURE EVALUATION.....	17
9. TEAM CONTRIBUTIONS.....	17
10. CONCLUSION AND REFLECTION.....	18

1. INTRODUCTION

This Cloud-Native Learning Management System (LMS) utilizing a Layered Architecture style is specifically engineered to address the critical infrastructure and performance challenges faced by modern educational platforms at Universiti Malaysia Terengganu (UMT). Traditional academic systems frequently suffer from severe latency, database lockups and complete runtime server failures during high-concurrency periods, such as course registration weeks and synchronized examination windows.

To eliminate these infrastructural single points of failure, the proposed platform implements a hybrid architectural model. It combines a structured Layered Architecture to isolate internal system concerns (such as UI presentation and data persistence) with Cloud-Native design principles where business capabilities operate as decoupled services. This comprehensive approach guarantees efficient traffic distribution, high availability, fault isolation, and seamless modular integration for all three primary user classes: Students, Lecturers, and System Administrators.

2. PROBLEM DESCRIPTION

a) Monolithic Architectural Rigidity

Current learning platforms typically utilize legacy monolithic frameworks where essential functionalities, including user authentication and grade management, are interdependent. Consequently, a single localized error may compromise the entire system, complicating maintenance and increasing the risk associated with updates.

b) Scalability & Latency Bottlenecks

Conventionally hosted infrastructures often lack the capacity for automated resource allocation. In periods of intensive concurrent activity, such as final examination windows, the inability to scale horizontally leads to significant latency, unresponsive services, and overall performance decline.

c) Integration and Accessibility Constraints

The lack of a modular API structure prevents seamless synchronization with external academic tools and media services. Furthermore, many outdated systems fail to provide a responsive, device-independent interface, which hinders the digital experience for students accessing resources via mobile hardware.

3. SYSTEM REQUIREMENTS

3.1 FUNCTIONAL REQUIREMENT

a. User Management

- FR - 1.1 : The system shall allow users to register an account by providing email, full name and password.
- FR - 1.2 : The system shall provide a secure login interface that authenticates users with encrypted credentials.
- FR - 1.3 : The system shall enforce Role Based Access Control, which ensures only authorized users can access specific pages such as the user list for admin.

b. Course Management

- FR - 2.1 : The system shall allow instructors to create new courses by entering a category, title and description.
- FR - 2.2 : The System shall support uploading course materials in multiple formats, including PDF documents and MP4 video files.
- FR - 2.3 : The system shall provide a search function that enables students to find courses using keywords or instructor names.

c. Assignment & Submission

- FR - 3.1 : The system shall allow instructors to set specific due dates and time limits for each assignment.
- FR - 3.2 : The system shall permit students to upload assignment files with a maximum size limit.
- FR - 3.3 : The system shall automatically block or reject a submission made after the deadline.

d. Notification

- FR - 4.1 : The system shall send automated notifications via email or dashboard when instructors post new course material.
 - FR - 4.2 : The system shall send a push notification to student 24 hours before an assignment deadline.
- e. Progress Tracking
- FR - 5.1 : The system shall update student a percentage complete each time they mark a lesson as finished.
 - FR - 5.2 : The system shall display a visual progress bar on the student dashboard for every enrolled course.

3.2 NON-FUNCTIONAL REQUIREMENT

- a. Availability
- NFR - 1.1 : The system shall maintain an uptime 99.9% ensuring student can reliably access course material and submit assignments at any time.
 - NFR - 1.2 : The system shall implement an automated failover mechanism to guarantee service continuity in the event of a cloud server failure.
- b. Scalability
- NFR - 2.1 : The system shall support horizontal scaling which enabling it to handle sudden increase in concurrent user without performance degradation.
 - NFR - 2.2 : The database shall be capable of storing and retrieving data for up to 10,000 active users while maintaining acceptable latency levels.
- c. Performance
- NFR - 3.1 : The system shall deliver a response time of less than 2 seconds for standard page loads under normal traffic conditions.
 - NFR - 3.2 : File upload and download processes shall be optimized like a 20MB assignment file is completed within 10 seconds.
- d. Security
- NFR - 4.1 : all sensitive data, include user password and personal information shall be encrypted or equivalent industry standard before storage in the database.

- NFR - 4.2 : The system shall enforce HTTP for all data transmission to safeguard against unauthorized interception of student and instructor information.

e. Maintainability & usability

- NFR - 5.1 : The system architecture shall be modular which allows for seamless updates or the addition of new features without disrupting existing services.
- NFR - 5.2 : The user interface shall be fully responsive by ensuring consistent functionality across smartphone, desktops and tablet,

4. ARCHITECTURE DESIGN

The system implements a hybrid Cloud-Native Layered Architecture powered by isolated domain Microservices.

No.	Services	Description
1	Presentation Layer	A responsive web interface built to execute across heterogeneous devices via standard web browsers without native application wrappers.
2	API Gateway Layer	Functions as the single, centralized reverse-proxy entry point for all incoming client traffic. It encapsulates backend microservice topology, orchestrating critical cross-cutting concerns like edge rate limiting, request validation, authentication token inspection, and intelligent load balancing.
3	Application (Microservices)	Core business logic is structurally decoupled

	Layer	into individual, stateless microservices (User Service, Course Service, Assignment Service, Notification Service, and Progress Service). These independent components converse over lightweight RESTful APIs or gRPC, ensuring full fault isolation.
4	Data Layer	Embraces the isolated <i>Database-per-Service</i> design paradigm. This completely eliminates structural coupling at the data storage level, avoiding single-database bottlenecks or transactional schema deadlocks during periods of peak load.

5. ARCHITECTURE DIAGRAMS

5.1 SYSTEM CONTEXT DIAGRAM

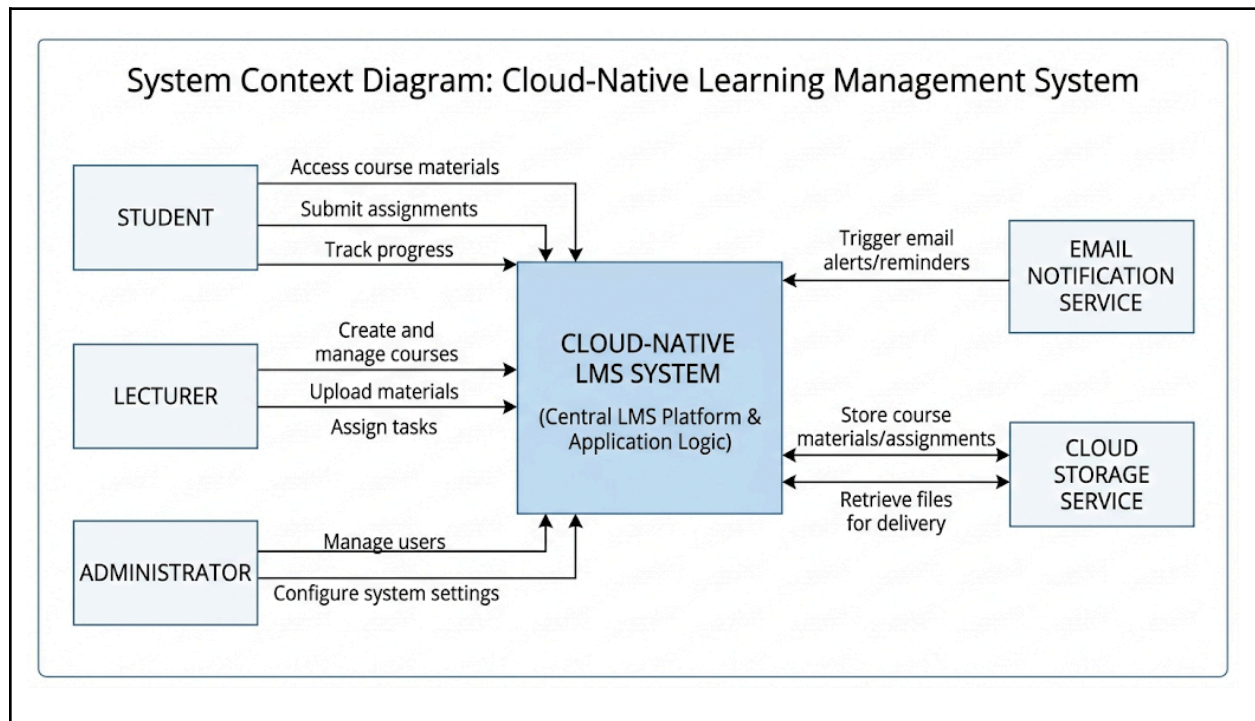


Figure 3 : System Context Diagram for Cloud-Native Learning Management System.

The System Context Diagram provides a high-level architectural abstraction of the external interface boundaries between the Cloud-Native LMS and its primary actor classes, specifically Students, Lecturers, and System Administrators. This visualization emphasizes the critical integration points with third-party infrastructure services, including distributed cloud storage and automated email notification engines. By encapsulating internal component complexity, the diagram illustrates the orchestrations of user interactions and external system dependencies essential to the platform's operational scope.

5.2 COMPONENT / SERVICES ARCHITECTURE DIAGRAM

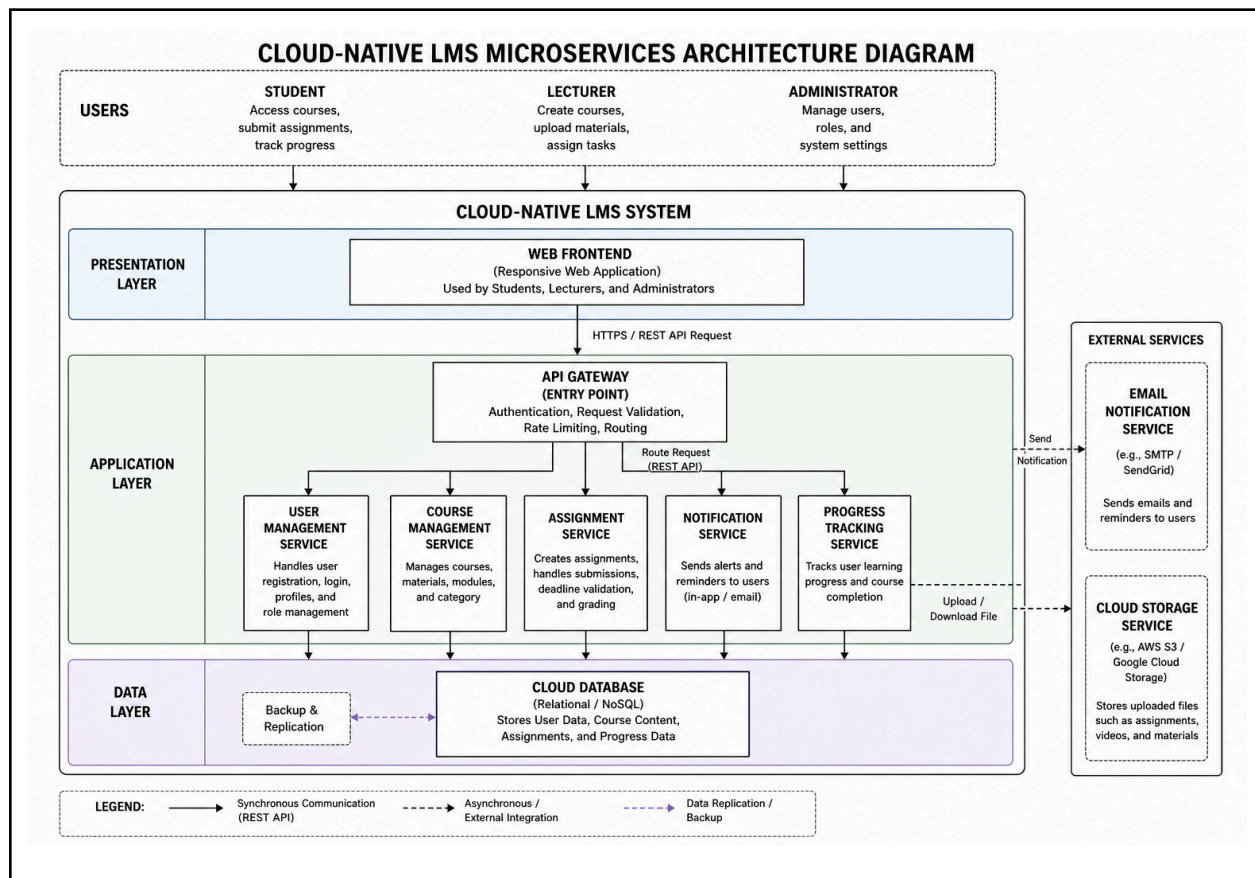


Figure 4 : Component Diagram for Cloud-Native LMS Microservices Architecture Diagram

The internal architectural configuration of the Cloud-Native LMS, predicated on a structured layered design, is visually represented within the Component/Microservices Architecture

Diagram. This framework is strategically partitioned into Presentation, Application, and Data layers, each assigned specific operational mandates to ensure cohesive system functionality.

Within the Presentation Layer, a responsive Web Frontend serves as the primary interface for Students, Lecturers, and Administrators. This segment orchestrates all user-centric interactions, ranging from course administrative tasks to assignment submissions and content retrieval. The frontend communicates with backend services via secure RESTful API queries over HTTPS to maintain data integrity and security.

The Application Layer functions as the system's computational core, where complex business logic is executed. A centralized API Gateway operates as the single entry point, managing edge concerns such as authentication, request validation, and intelligent routing. The architecture utilizes decoupled microservices—specifically the *User Service*, *Course Service*, *Assignment Service*, *Notification Service*, and *Progress Service*—which can be independently scaled and maintained, thereby enhancing overall system modularity.

The Data Layer utilizes a robust Cloud Database to manage all persistent records, including user credentials, academic content, and performance tracking. This centralized persistence strategy is engineered to guarantee high availability and efficient data orchestration across the entire platform.

Furthermore, the platform leverages external infrastructure integrations to bolster operational capabilities. The Assignment Service interfaces with distributed Cloud Storage for optimized file handling, while the Notification Service synchronizes with an automated Email Engine to deliver timely academic alerts and reminders to users.

In summary, the proposed architecture exemplifies a scalable and maintainable engineering model. By synthesizing microservice-oriented design with cloud-native principles, the system is uniquely positioned to satisfy the rigorous performance and accessibility demands of contemporary digital learning environments.

5.3 INTERACTION DIAGRAM

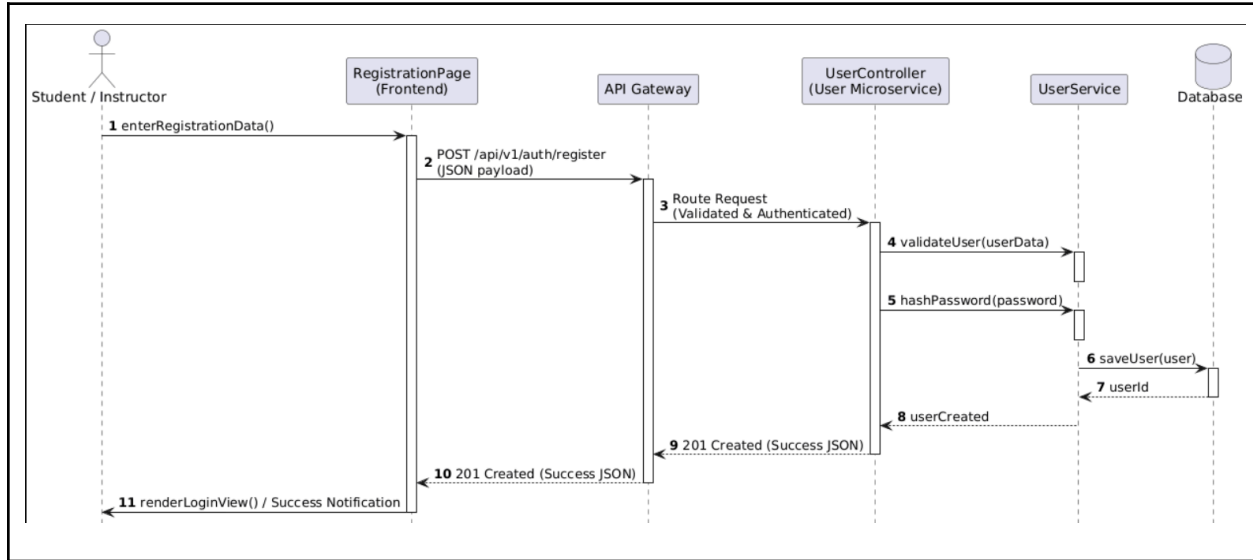


Figure 5 : User Management (User Registration) Sequence Flow

The cloud-native sequence diagram for the User Registration module illustrates a fully decoupled, API-first communication lifecycle. The process initiates when a Student or Instructor submits credential inputs through the responsive RegistrationPage presentation interface. This client payload is dispatched as a synchronous HTTPS POST network request to the API Gateway. The gateway executes edge validation and policies before securely proxy-routing the JSON payload to the UserController inside the independent User Management Microservice. The internal UserService validates the entity structure , cryptographically hashes the raw text password strings , and commits the user state object to the isolated Database domain. Upon generating a success response code (201 Created) , the confirmation cascades back through the gateway layer to the client-side UI, triggering a state update to prompt the user to log in safely.

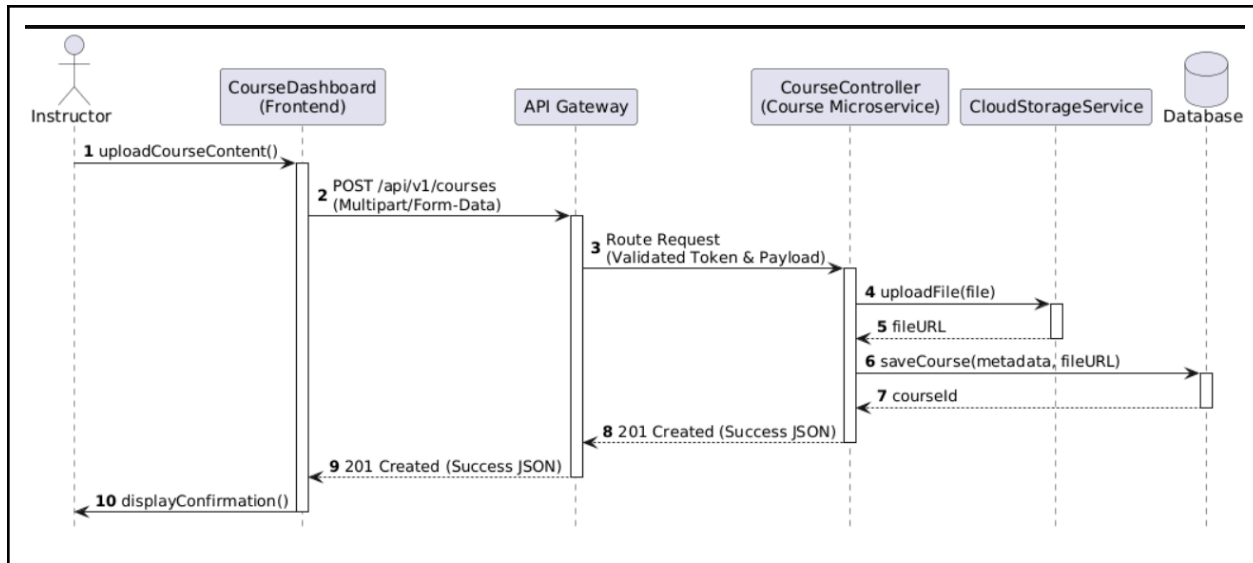


Figure 6: Course Management (Create New Course) Sequence Flow

The architectural sequence for the Course Management module demonstrates a decoupled, event-driven lifecycle for the ingestion of academic assets. This process commences when an *Instructor* submits media files and curricular metadata through the responsive *CourseDashboard* presentation layer. This client-side interaction is dispatched as a secure HTTPS multipart request to the centralized API Gateway, which performs edge orchestration including request throttling and identity verification. The gateway then proxy-routes the network payload to the *CourseController* situated within the isolated Course Management Microservice. This service component offloads binary content to distributed Cloud Storage for optimized persistence while retaining a unique resource locator string. This metadata is subsequently committed to the domain-specific Database to ensure state consistency. Upon receipt of a transactional success code, the confirmation signal cascades back to the frontend interface, triggering a visual status update to notify the user of a successful upload.

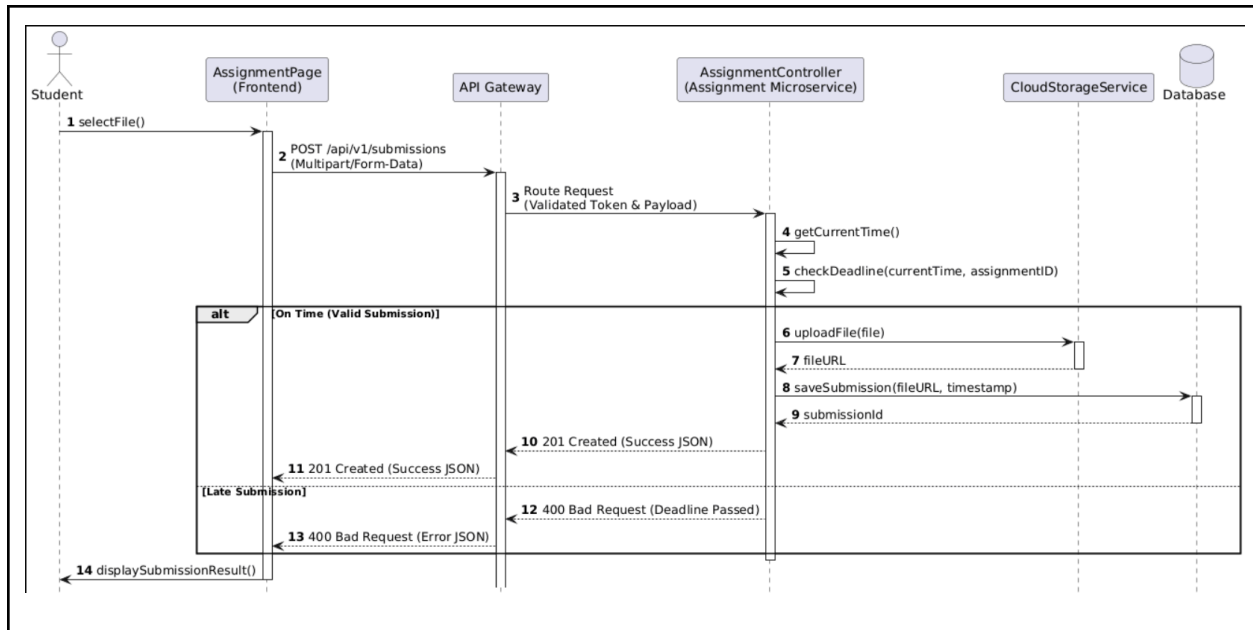


Figure 7: Assignment & Submission (Student Upload) Sequence Flow

The architectural sequence for the Assignment & Submission module demonstrates a decoupled, asynchronous lifecycle for the ingestion and verification of student artifacts. This process commences when a *Student* submits digital files through the responsive *AssignmentPage* presentation layer. This client-side interaction is dispatched as a secure HTTPS multipart request to the centralized API Gateway, which performs edge orchestration including traffic policy enforcement. The gateway then proxy-routes the network payload to the *AssignmentController* situated within the isolated Assignment Management Microservice. This service component executes an immediate temporal validation, comparing the current system timestamp against the curricular deadline metadata cached in memory. Upon validation, the controller offloads binary content to distributed Cloud Storage for optimized persistence while retrieving a unique resource locator string. This metadata, alongside the submission record, is subsequently committed to the domain-specific Database to ensure state consistency. Upon receipt of a transactional success code, the confirmation signal cascades back through the gateway to the frontend interface, triggering a visual status update to notify the user of a successful submission.

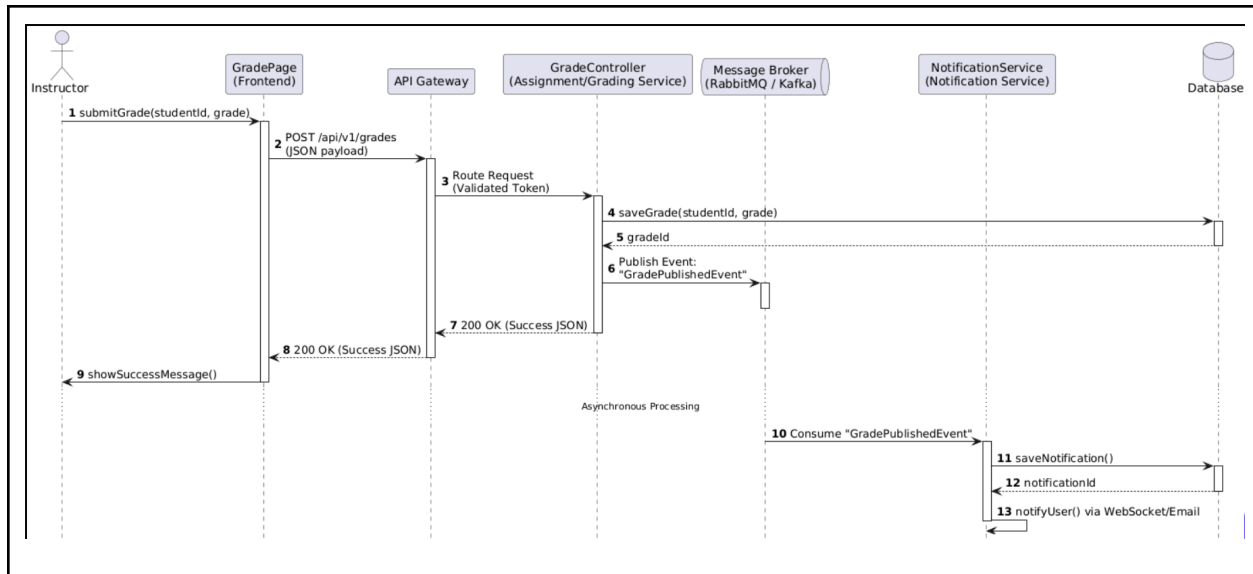


Figure 8: Notification Module (Trigger Grade Alert) Sequence Flow

The cloud-native sequence diagram for the Notification Module illustrates an asynchronous, event-driven orchestration pattern engineered for fault isolation. The process initiates when an Instructor dispatches a student evaluation evaluation score via the GradePage presentation layer. The interface submits an HTTP POST payload through the API Gateway, which validates and routes the traffic to the GradeController inside the independent Assignment/Grading Microservice. Once the record is successfully committed to the database , the grading service publishes a stateless GradePublishedEvent message packet to the distributed Message Broker (RabbitMQ/Kafka) and immediately delivers a successful execution response back to the client interface. This non-blocking pattern decouples the services completely; the NotificationService independently consumes the event payload from the message queue asynchronously, persists the alert configuration to the data layer, and handles real-time delivery channels (such as WebSockets or email push mechanisms) to the end user without impacting the core processing metrics of the grading application layer.

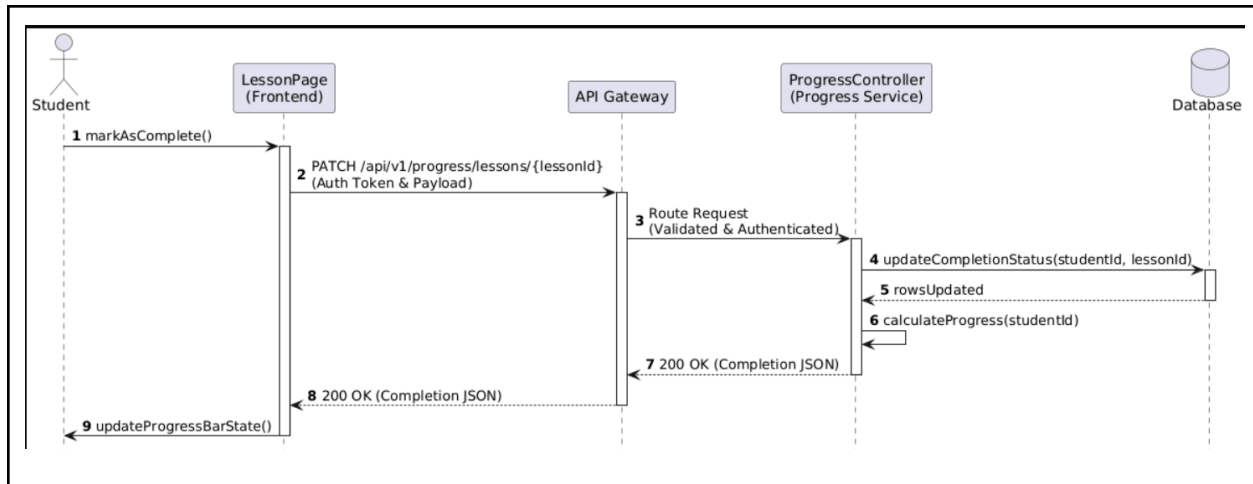


Figure 9: Process Tracking (Update Completion) Sequence Flow

The cloud-native sequence diagram for the Progress Tracking module outlines the runtime lifecycle of a user state completion transaction. The process initiates when a Student interacts with the responsive LessonPage client layer to trigger a completion state change. This client event is transformed into an asynchronous HTTP PATCH request and dispatched directly to the API Gateway. The gateway enforces authorization validation filters before routing the request packet to the ProgressController inside the decoupled Progress Tracking Microservice. The service triggers an isolated transaction against the domain Database to record the lesson completion status. Upon processing the update, the microservice internal logic handles the aggregation metrics to compute the student's total updated course progress metrics. This calculated state is returned as a structured JSON object inside a standard HTTP 200 OK network response, cascading back to the presentation layer where the frontend state manager dynamically repaints the visual progress dashboard components without requiring a full page refresh.

5.4 DEPLOYMENT DIAGRAM

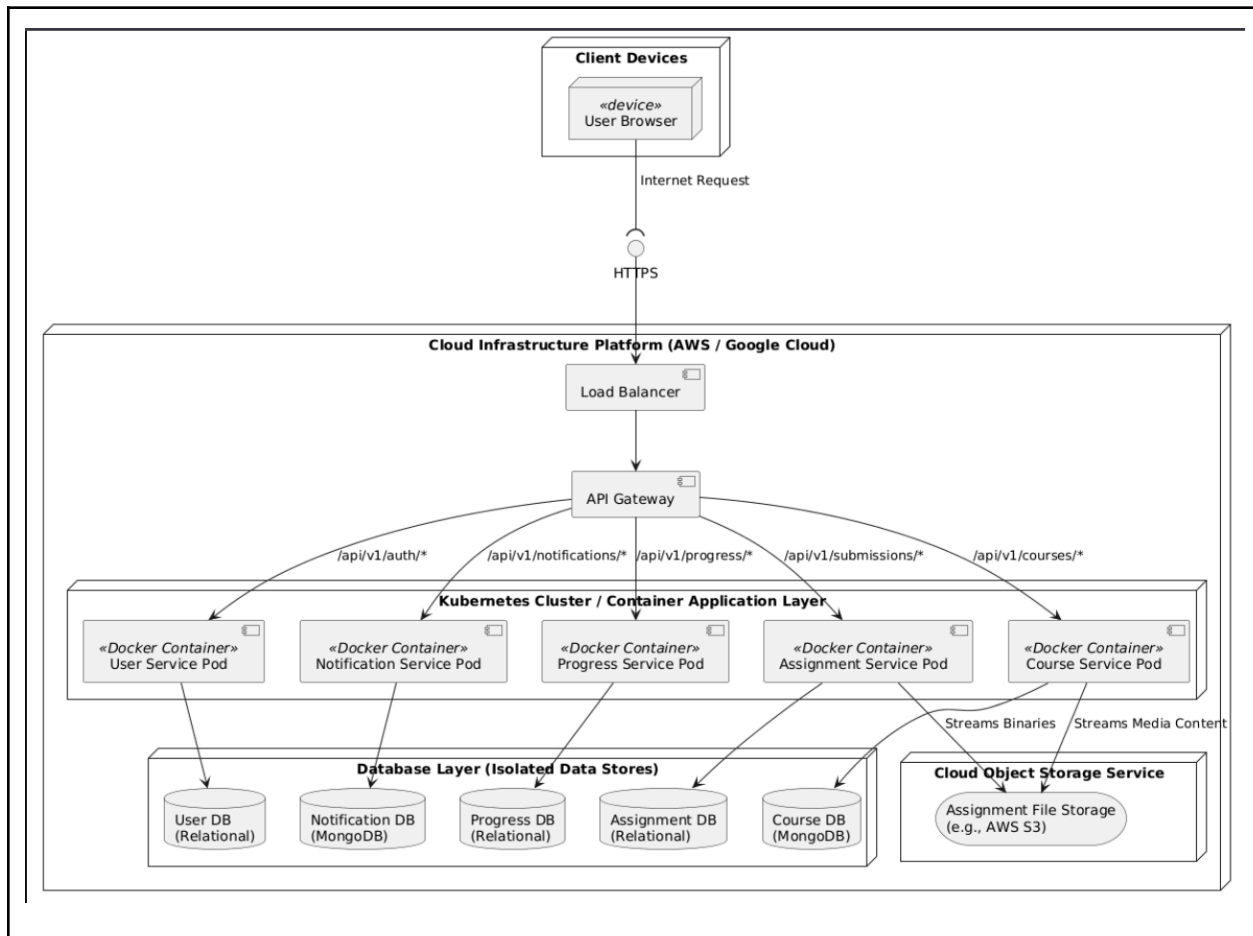


Figure 10: Deployment Diagram of Cloud-Native Learning Management System (LMS)

The deployment diagram maps out the physical distribution of the LMS system components across an elastic cloud infrastructure ecosystem. At the edge client boundary, end-users interface with the responsive web application utilizing standard modern web browsers on cross-platform consumer per per-devices. Inbound public application traffic is secured over HTTPS and received by a high-availability Load Balancer, which evenly disperses concurrent call requests to prevent runtime server saturation. The filtered stream hits the API Gateway layer for edge token decryption, request mapping, and localized reverse-proxy routing. Within the container orchestration application layer, each system domain operates as independent, stateless microservice units encapsulated inside immutable Docker Containers. To prevent tight-coupling architectural anomalies, the data layer strictly enforces the Database-per-Service pattern, allocating independent database nodes to completely separate data schema access. Blob assets

are stored via a dedicated Cloud Object Storage Service, allowing the microservices to process lightweight JSON records independently while remaining horizontally scalable, robust, and completely fault-tolerant.

6. TECHNOLOGY STACK

a) Frontend UI Framework

React.js combined with Tailwind CSS to build highly responsive, performance-optimized, and device-agnostic user interfaces.

b) API Gateway & Edge Proxy

Spring Cloud Gateway or Kong Gateway to serve as the unified reverse proxy, managing security filters and traffic policing.

c) Application Services (Backend)

Node.js (Express) or Spring Boot running as lightweight, stateless containerized workloads that expose modular RESTful JSON APIs.

d) Containerization & Orchestration

Docker to package every individual service alongside its unique environment dependencies, with Kubernetes orchestrating cluster scaling, load scheduling, and automated self-healing.

e) Database Management System

PostgreSQL/MySQL to handle strict transactional database states (such as user records, permissions, and grades), paired with MongoDB (NoSQL) for storing highly flexible, unstructured course curricula and rich learning content metadata.

7. PROTOTYPE IMPLEMENTATION

This section presents the visual representation of the developed system, demonstrating the functionality and user interface design of the platform.

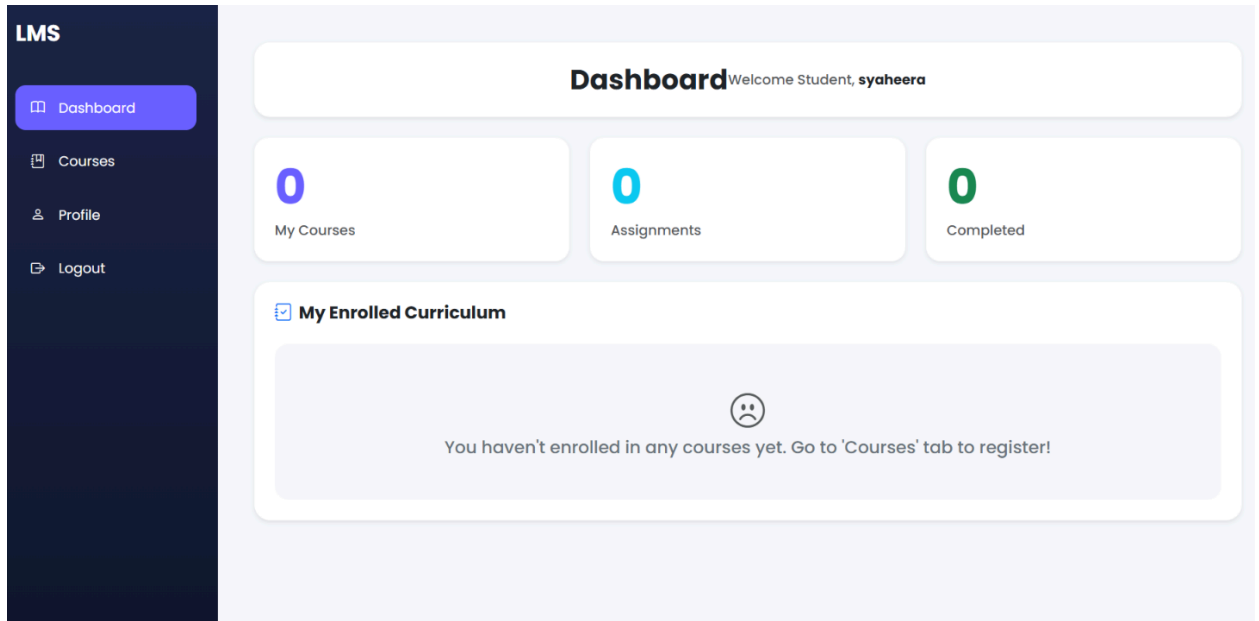


Figure 7.1: Student Dashboard Interface

The Student Dashboard acts as the primary user interface (UI) designed for personalized learning management. It integrates real-time data from the enrollments, assignments, and submissions tables in the `lms_db` database. The visual hierarchy is structured to prioritize user productivity, where the 'My Courses' count provides an immediate snapshot of the student's current workload. This section utilizes server-side rendering (JSP) to fetch data dynamically based on the student's session ID, ensuring that the information displayed is always specific and relevant to the logged-in user.

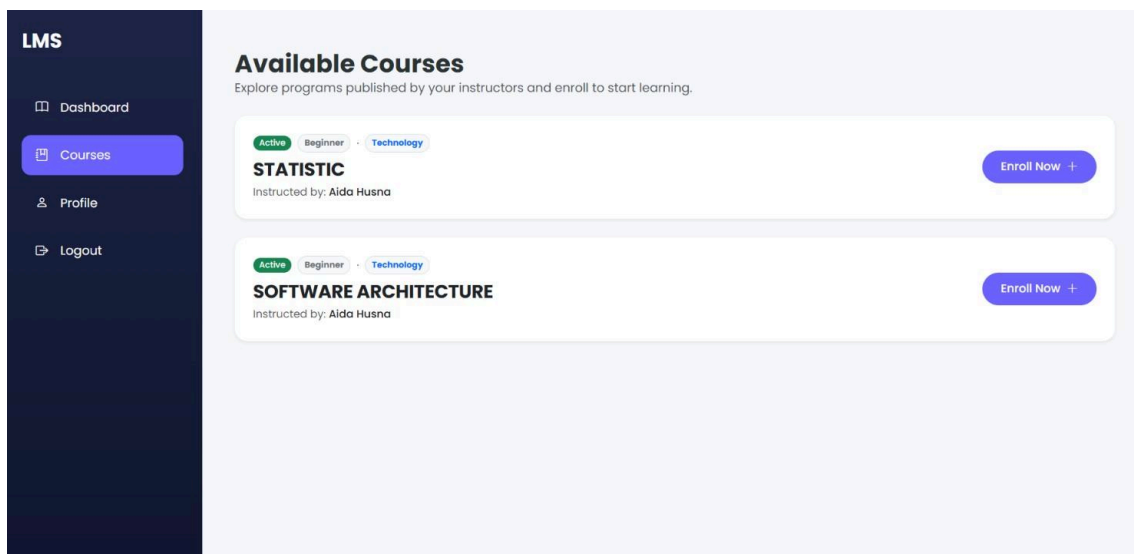


Figure 7.2: Course Selection and Enrollment

The Course Selection page implements a dynamic filtering mechanism that allows students to browse through the curriculum. Behind the scenes, the enrollment process utilizes a doGet servlet handler that processes the courseId and studentId parameters. This figure highlights the user's journey from course discovery to registration. The implementation includes server-side validation to prevent duplicate enrollments, ensuring database integrity and providing immediate visual feedback to the student via URL parameters upon a successful transaction

The figure displays two screenshots of the LMS (Learning Management System) interface. The top screenshot shows the 'Register' page, which includes a form for user registration. The bottom screenshot shows the 'Welcome Back!' login page, which includes a form for user login and a 'Login' button.

Register Page:

- LMS**
- Register**
- Register to continue
- No. Matrik:
- Full Name:
- Username:
- Email:
- No. Telefon:
- Password:

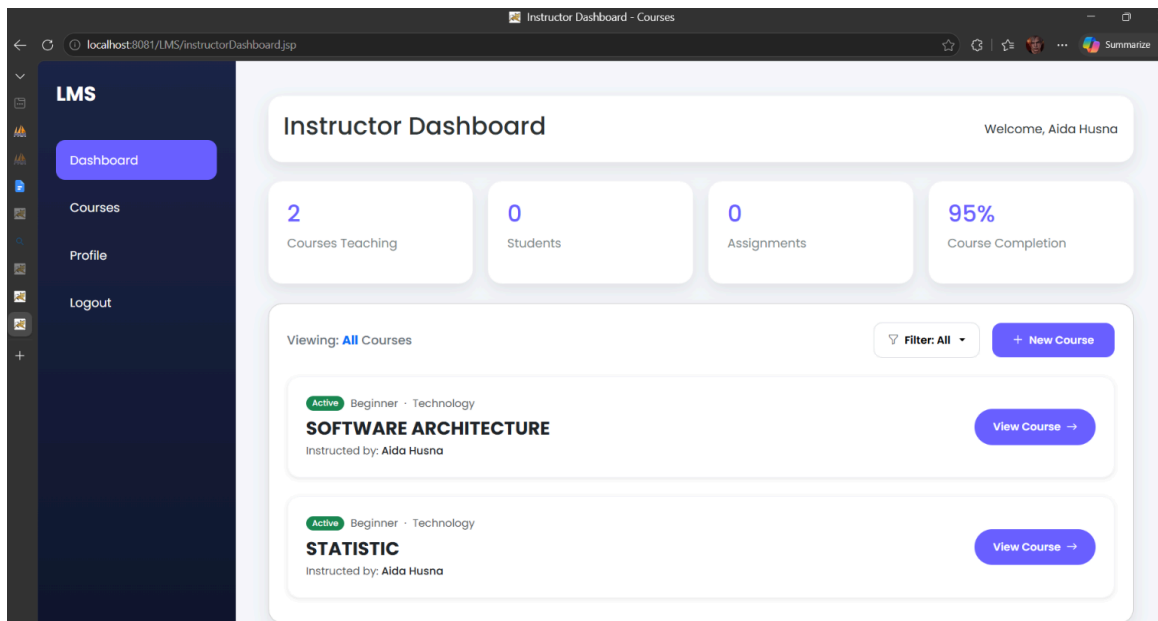
Welcome Back! Page:

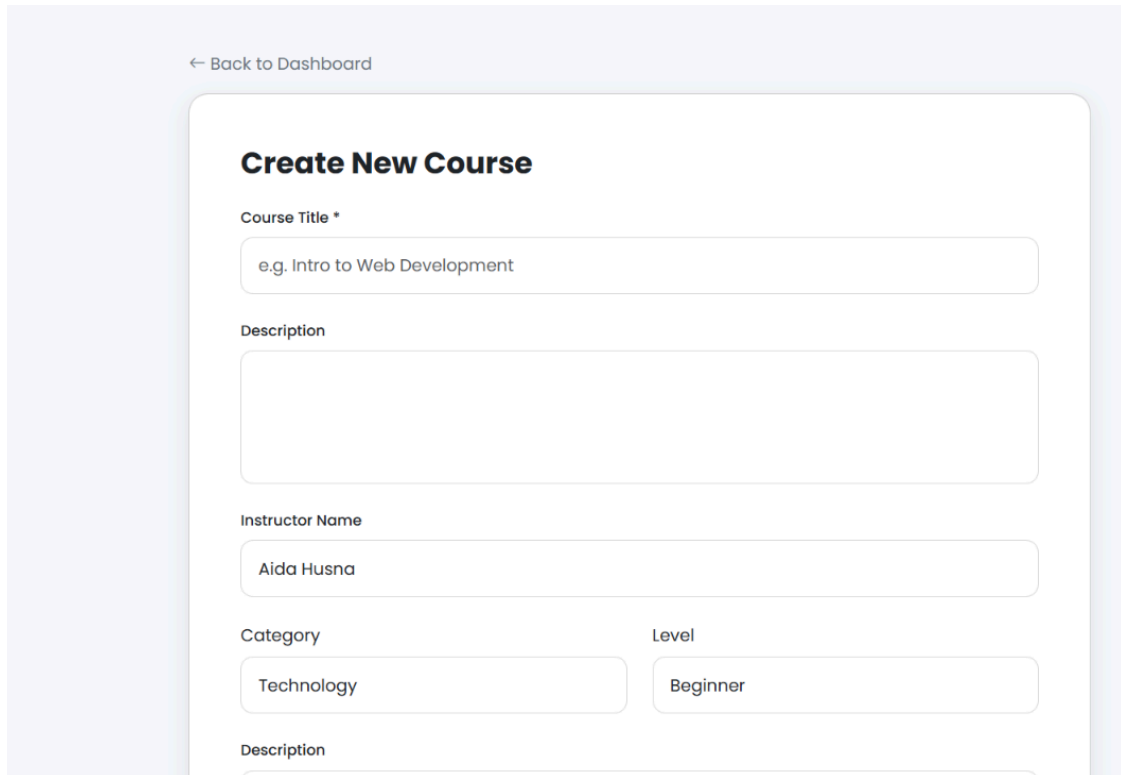
- LMS**
- Welcome Back!**
- Please login to your account
- No. Matrik:
- Password:
- Log in As:
- Login**
- Don't have an account? [Register here](#)

The bottom screenshot also features an illustration of a person's head with gears and a ladder, symbolizing learning and problem-solving.

Figure 7.3: User Authentication (Login/Register)

The authentication module is the gateway to the system, ensuring that access is restricted to authorized personnel only. This figure displays the front-end forms for both registration and login, which are connected to the RegisterServlet and LoginServlet respectively. The logic flow utilizes a UserDao class to handle secure database interactions. By separating the roles into 'Student', 'Instructor', and 'Admin', the system enforces role-based access control (RBAC) at the servlet level, verifying user credentials against the users table before granting access to the respective dashboards.





The screenshot shows a web interface for creating a new course. At the top left, there is a link labeled "← Back to Dashboard". The main heading is "Create New Course". Below this, there are several input fields: "Course Title *" with a placeholder "e.g. Intro to Web Development", "Description" (a large text area), "Instructor Name" with the value "Aida Husna", "Category" with the value "Technology", and "Level" with the value "Beginner". There is also a partially visible "Description" field at the bottom.

← Back to Dashboard

Create New Course

Course Title *

e.g. Intro to Web Development

Description

Instructor Name

Aida Husna

Category

Technology

Level

Beginner

Description

Figure 7.4: Instructor Dashboard & Course Management

This figure illustrates the Instructor Dashboard, a centralized control panel for managing academic content. The instructor is provided with the capability to create, update, and delete courses. When an instructor clicks 'Add Course', the data is processed by `CreateCourseServlet` and stored in the courses table within the `lms_db` database. This module demonstrates the system's dynamic content management capability, allowing instructors to maintain up-to-date course materials without requiring manual database intervention.

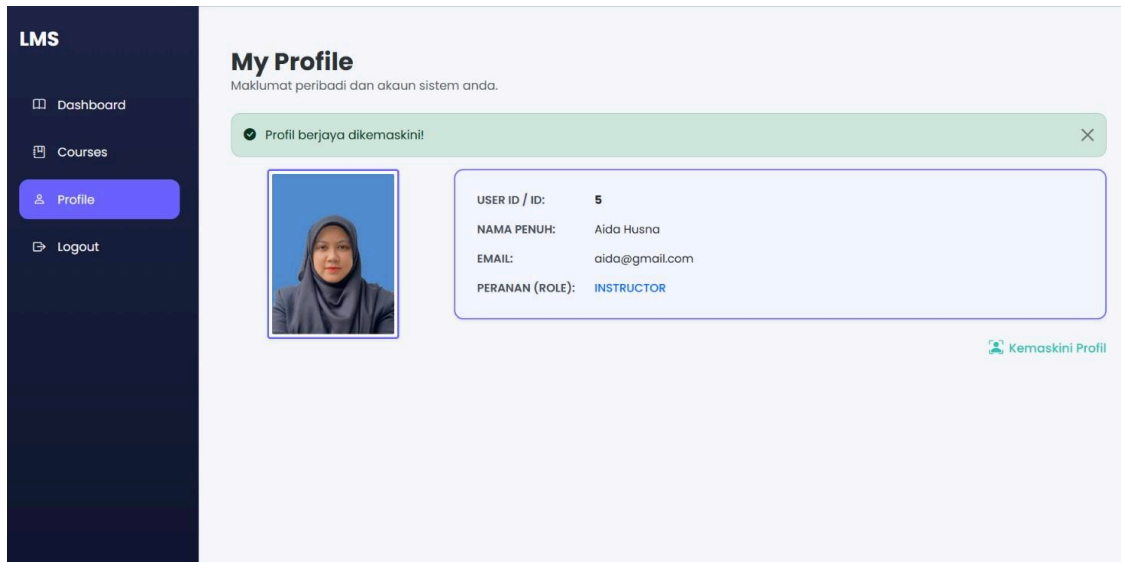


Figure 7.5: User Profile View

The User Profile view provides a personalized summary of the student or instructor's details. The data displayed is retrieved dynamically from the users table based on the primary key, id (Matric Number). This figure demonstrates the system's capability for data persistence, ensuring that user-specific information remains accurate throughout the session.

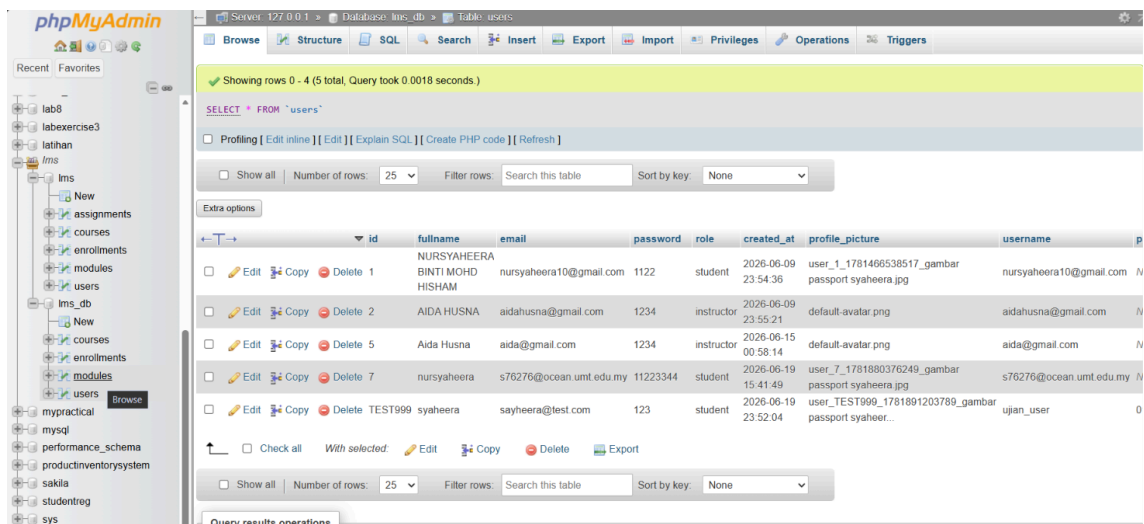


Figure 7.6: Backend Logic & Database Integration

This figure captures the core logic layer of the application, specifically the DAO (Data Access Object) classes. By employing the Model-View-Controller (MVC) pattern, we isolate the database connection logic from the presentation layer. The snippet highlights

the use of PreparedStatement to execute INSERT and SELECT queries, which acts as a secure 'bridge' between the Java Servlet controllers and the MySQL database. This architecture ensures that the application is modular, secure, and easier to debug

8. ARCHITECTURE EVALUATION

The system architecture was evaluated based on its performance, reliability, and ability to handle the specific requirements of an academic learning management environment.

Criteria	Evaluation
Scalability	The use of MySQL as a relational database allows for stable data growth. As student records increase, the normalized structure ensures consistent performance.
Reliability	The use of Railway (PaaS) for the application layer enables automated deployments and self-healing properties, ensuring the LMS remains operational with minimal downtime.
Integrity	By employing SQL PreparedStatements, the architecture effectively prevents SQL injection attacks, ensuring that user data, grades, and permissions remain secure.
Cloud-Native Storage	Integration with AWS services provides enterprise-grade durability, automated backups, and geo-redundancy, safeguarding data against hardware failures.
Infrastructure	The combination of Railway for rapid application delivery and AWS for robust database management optimizes the CI/CD pipeline, reducing manual configuration overhead.

The system's cloud-native architecture leverages Railway to decouple application logic from infrastructure, enabling independent scaling and rapid deployment cycles while maintaining high stability. Complementing this, AWS provides a robust storage layer that offloads critical maintenance such as security patching and disaster recovery, ensuring consistent performance even during traffic spikes. By utilizing an MVC pattern for modularity and maintainability, the architecture successfully delivers a stable, transactional-based system that is both secure and highly extensible for future integration with NoSQL databases.

9. TEAM CONTRIBUTIONS

Table 1 : Team Members Roles & Responsibilities

Table 1 presents the roles assigned to each team member along with their respective responsibilities throughout the project development.

No.	Name	Role	Responsibilities
1.	Nur Mas Qasrina Binti Mas Jaya	Project Manager / Software Architect	Directs the comprehensive project framework, architecting high-level cloud-native and layered system configurations. This role involves modeling complex component relationships and managing the unified integration of all services.
2.	Aida Husna Binti Muhamad Zaidi	Backend & Database Engineer	Designs independent database schemas at the Data Layer, implements stateless business logic API controllers at the Application Layer, configures edge security filters, and handles Docker container builds.
3.	Nursyaheera Binti Mohd Hisham	Frontend & UI/UX Engineer	Develops the device-agnostic, responsive client-side interface at the Presentation Layer, handles client application state management, integrates network fetch hooks to the API gateway, and optimizes cross-browser rendering.

10. CONCLUSION AND REFLECTION

Transitioning from traditional monolithic design patterns to a decoupled, Cloud-Native Layered Architecture is the optimal engineering strategy to permanently mitigate the platform instability typically found in university-wide LMS platforms. This project provides foundational software engineering insights into distributed service lifecycle synchronization, API contract modeling, and strict data layer isolation patterns. The finalized architecture successfully ensures that UMT students and faculty members can depend on a highly available, robust, and future-ready digital learning space.